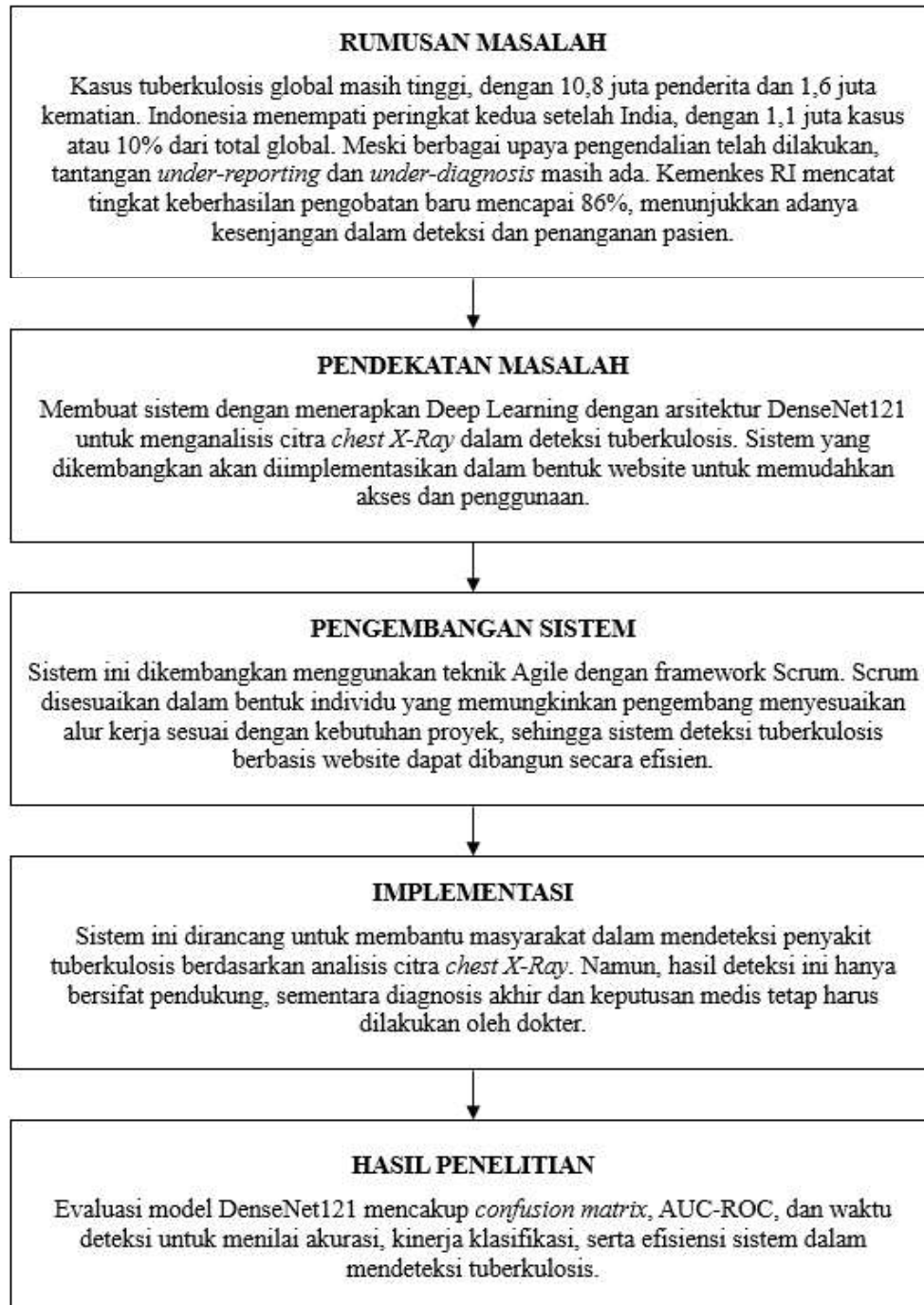


C. Kerangka Berpikir



Gambar 2.16 Kerangka Berpikir

BAB III

METODE PENELITIAN

A. Tempat dan Waktu Penelitian

Penelitian ini dilakukan di Laboratorium Komputer Kampus III Universitas PGRI Madiun, yang beralamat di Jalan Auri No. 14-16, Kota Madiun, Jawa Timur. Penelitian ini dimulai pada tanggal 10 Maret 2025, bertepatan dengan terbitnya Surat Keputusan (SK) dosen pembimbing, dan berakhir pada tanggal 10 Juni 2025. Rincian jadwal penelitian dapat dilihat pada Tabel 3.1 berikut.

Tabel 3.1 Waktu dan Tahapan Penelitian (Masih Estimasi Perencanaan)

No	Keterangan	Tahun 2025															
		Maret				April				Mei				Juni			
		1	2	3	4	1	2	3	4	1	2	3	4	1	2	3	4
1	Studi Literatur																
2	Pengumpulan Data																
3	Preprocessing Data																
4	Perancangan Model																
5	Evaluasi Model																
6	Perancangan Sistem																
7	Implementasi Sistem																
8	Pengujian Sistem																
9	Publikasi																

Keterangan:



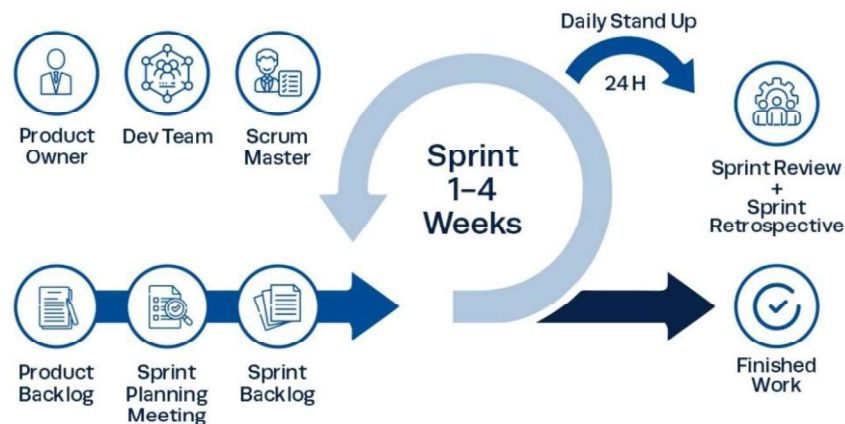
: Ada Kegiatan



: Tidak Ada Kegiatan

B. Metode Pengembangan Sistem

Penelitian ini menggunakan metode pengembangan Agile Framework Scrum dengan penyesuaian individu. Scrum, yang biasanya melibatkan tim, diadaptasi agar dapat diterapkan oleh satu pengembang. Seluruh peran dalam Scrum, seperti Product Owner, Scrum Master, dan Tim Pengembang, dijalankan oleh satu individu. Tahapan pengembangan sistem pada penelitian ini dapat dilihat pada Gambar 3.1.



Gambar 3.1 Agile Framework Scrum

Berdasarkan proses pada Gambar 3.1, Agile Framework Scrum memiliki alur kerja iteratif dalam pengembangan perangkat lunak dengan siklus sprint yang berlangsung selama 1–4 minggu. Berikut tahapan utama dalam proses Scrum:

1. Product Backlog

Product Backlog merupakan daftar fitur atau tugas yang perlu dikembangkan dalam sistem. Daftar ini disusun dan dikelola oleh Product Owner berdasarkan prioritas bisnis serta kebutuhan pengguna.

2. Sprint Planning Meeting

Pada tahap ini, tim pengembang bersama Product Owner memilih item dari Product Backlog yang akan dikerjakan dalam satu sprint. Hasil dari pertemuan ini adalah Sprint Backlog, yang berisi tugas-tugas yang akan dikerjakan selama periode sprint.

3. Sprint Backlog

Sprint Backlog adalah sekumpulan tugas yang telah dipilih dari Product Backlog untuk dikerjakan dalam sprint yang sedang berlangsung. Tugas-tugas ini dikelola oleh tim pengembang dan menjadi pedoman kerja selama sprint.

4. Sprint Execution & Daily Stand-up

Dalam tahap ini, tim pengembang mulai mengerjakan tugas sesuai Sprint Backlog. Setiap hari, dilakukan Daily Stand-up (Daily Scrum) untuk membahas progres, hambatan, serta perencanaan kerja selanjutnya dalam sprint. Daily Stand-up biasanya dilakukan dalam waktu singkat (sekitar 15 menit).

5. Sprint Review & Sprint Retrospective

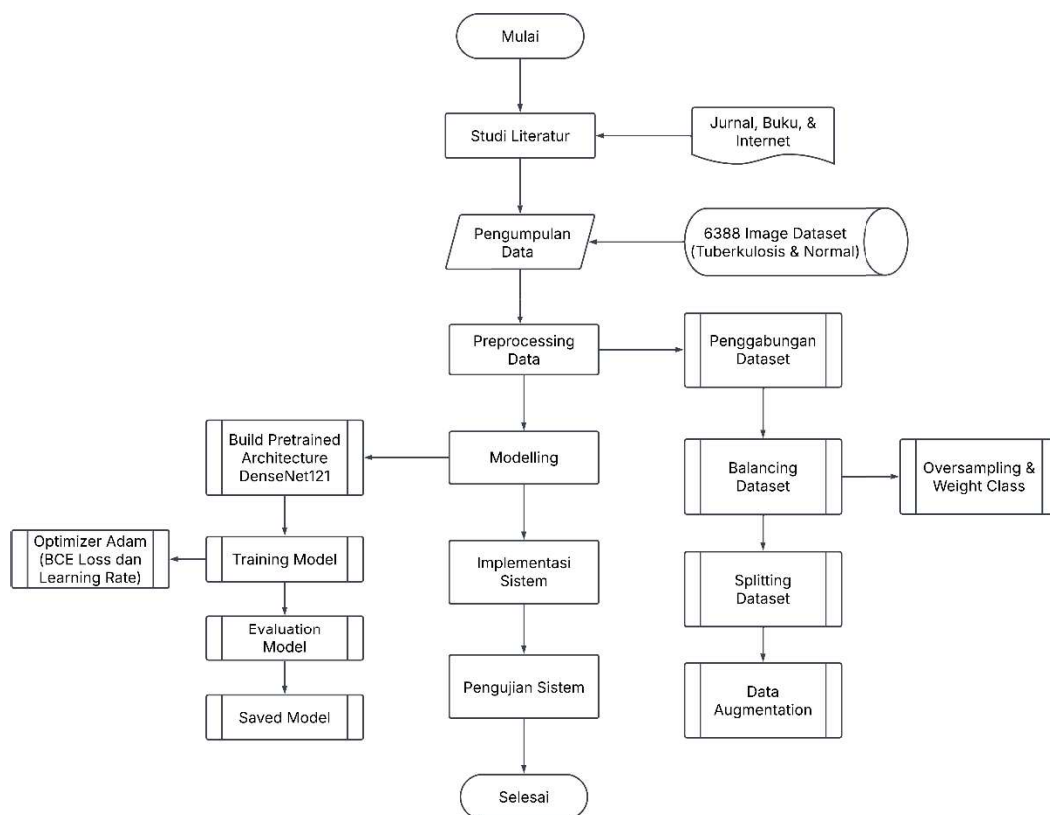
Setelah sprint selesai, dilakukan Sprint Review untuk mengevaluasi hasil kerja dan mendemonstrasikan fitur yang telah dikembangkan. Kemudian, dalam Sprint Retrospective, tim melakukan refleksi untuk mengidentifikasi perbaikan yang bisa diterapkan pada sprint berikutnya.

6. Finished Work

Tahapan ini menandai bahwa backlog yang telah dikerjakan dalam sprint telah selesai dan dianggap sebagai Increment, yaitu versi produk yang telah diperbarui dan siap digunakan atau diuji lebih lanjut.

C. Rancangan Penelitian

Pada bagian ini akan membahas rancangan penelitian yang digunakan dalam pengembangan sistem klasifikasi Tuberkulosis berbasis Deep Learning. Untuk memberikan gambaran yang lebih jelas, tahapan penelitian divisualisasikan dalam bentuk *flowchart* yang menunjukkan proses penelitian dari tahap awal hingga tahap evaluasi pada Gambar 3.2 berikut.



Gambar 3.2 Flowchart Perancangan Penelitian

Flowchart pada Gambar 3.2 menggambarkan tahapan penelitian secara sistematis. Setiap langkah dalam penelitian ini akan dijelaskan lebih lanjut pada pembahasan berikut.

1) Studi Literatur

Studi literatur dilakukan untuk mengumpulkan dan menganalisis berbagai referensi yang relevan, seperti jurnal, buku, dan penelitian sebelumnya terkait deteksi Tuberkulosis menggunakan Deep Learning. Tahap ini bertujuan untuk memahami pendekatan yang telah digunakan serta menentukan metode yang sesuai untuk penelitian ini.

2) Pengumpulan Data

Pada penelitian ini, data yang digunakan merupakan data sekunder, yaitu data yang telah dikumpulkan dan dipublikasikan oleh pihak lain untuk keperluan penelitian sebelumnya. Data ini diperoleh dari dua sumber utama yaitu:

a. Tuberculosis (TB) Chest X-ray Database

Dataset ini berasal dari platform Kaggle yang dikategorikan menjadi positif TB dan normal. Dataset ini dikembangkan oleh tim peneliti dari Qatar University, University of Dhaka, dan beberapa institusi lainnya. Dataset ini mencakup 700 gambar X-ray TB dan 3500 gambar X-ray normal yang mempunyai resolusi gambar adalah 512×512 piksel. Dataset ini telah digunakan dalam berbagai penelitian karena jumlah datanya cukup besar

dan telah banyak digunakan dalam pengembangan model deep learning untuk deteksi tuberkulosis

b. Dataset of Tuberculosis Chest X-rays Images

Dataset ini berasal dari platform Mendeley Data yang dikategorikan menjadi positif TB dan normal. Dataset ini dikumpulkan dari rumah sakit lokal yang berada di Pakistan. Dataset ini mencakup 2494 gambar X-ray TB dan 514 gambar X-ray normal yang memiliki resolusi gambar adalah 256×256 piksel.

3) Preprocessing Data

a. Penggabungan Data

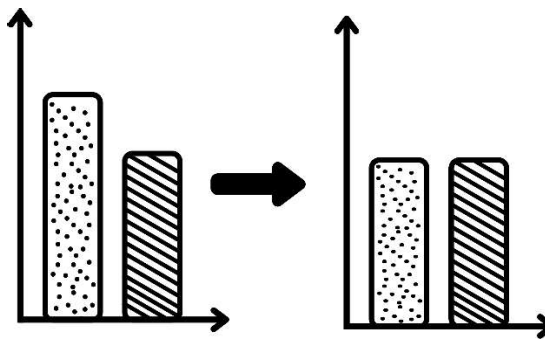
Pada tahap awal preprocessing, dilakukan penggabungan dataset dari dua sumber berbeda untuk memperoleh jumlah data yang lebih besar dan beragam. Dataset pertama berasal dari Kaggle, yang terdiri dari 700 gambar X-ray pasien TB dan 3500 gambar X-ray Normal dengan resolusi 512×512 piksel. Sementara itu, dataset kedua berasal dari Mendeley Dataset, yang berisi 2494 gambar X-ray pasien TB dan 514 gambar X-ray Normal dengan resolusi 256×256 piksel.

Setelah kedua dataset digabungkan, total data yang tersedia adalah 3194 gambar TB dan 4014 gambar Normal. Pada tahap selanjutnya, dilakukan balancing dataset untuk menangani ketidakseimbangan kelas sebelum proses data augmentation dan pelatihan model.

b. Balancing Dataset

Ketidakseimbangan data dalam klasifikasi dapat menjadi permasalahan yang signifikan, terutama karena model cenderung lebih sering memprediksi kelas dengan jumlah sampel terbanyak. Hal ini dapat menyebabkan model kurang memperhatikan kelas dengan jumlah data yang lebih sedikit, sehingga performa klasifikasi menjadi tidak optimal. Perbedaan jumlah sampel yang mencolok antara kelas mayoritas dan minoritas dapat memengaruhi akurasi model secara keseluruhan.

Untuk mengatasi hal tersebut, dalam penelitian ini dilakukan penyeimbangan jumlah data sebelum proses pembagian dataset menjadi data latih, validasi, dan uji. Teknik yang diterapkan dalam penelitian ini adalah undersampling pada kelas mayoritas serta class weighting pada fungsi loss.



Gambar 3.3 Undersampling

Pada tahap undersampling, jumlah sampel pada kelas mayoritas (Normal) dikurangi secara acak hingga seimbang dengan jumlah sampel kelas minoritas (TB), menghasilkan total 3194 gambar TB dan 3194 gambar

Normal. Langkah ini bertujuan agar model tidak terlalu terpengaruh oleh dominasi data dari kelas mayoritas.

Selain itu, untuk mengatasi kemungkinan bias yang masih tersisa, diterapkan metode class weighting. Bobot kelas dihitung berdasarkan proporsi jumlah sampel pada masing-masing kelas menggunakan rumus:

$$Class\ Weight = 1 - \frac{Jumlah\ Sample\ Kelas}{Total\ Sample} \quad (3.1)$$

Bobot ini kemudian diterapkan dalam fungsi loss selama pelatihan model guna memastikan bahwa model tetap mempertimbangkan kelas dengan jumlah data yang awalnya lebih sedikit. Setelah proses keseimbangan data ini dilakukan, dataset selanjutnya akan diproses lebih lanjut melalui pembagian dataset (data splitting) serta data augmentation sebelum memasuki tahap modeling.

c. Splitting Dataset

Setelah dilakukan balancing pada dataset, langkah selanjutnya adalah membagi dataset menjadi tiga bagian utama, yaitu training set, validation set, dan test set. Pembagian ini penting untuk melatih model, mengoptimalkan hyperparameters selama proses validasi, dan menguji performa model pada data yang belum pernah dilihat sebelumnya.

Dalam kasus ini, dataset dibagi dengan proporsi 70% untuk training set, 15% untuk validation set, dan 15% untuk test set. Proses ini dimulai dengan membagi dataset menjadi training set dan temporary set

(30%), kemudian sisa data (30%) dibagi lagi menjadi dua bagian yang sama besar untuk validation dan test set, masing-masing 15%.

Pembagian ini dilakukan menggunakan metode *train_test_split* dari sklearn, dengan stratify pada label agar distribusi kelas tetap seimbang di setiap subset. Setelah dataset terpisah, gambar-gambar tersebut disalin ke dalam folder masing-masing di dalam direktori splitting, yaitu *train*, *val*, dan *test*, yang memudahkan pengelolaan dan pemrosesan data lebih lanjut.

d. Data Augmentation

Data augmentation adalah proses teknik untuk memperbesar keragaman data pelatihan dengan mengaplikasikan beberapa transformasi pada gambar tanpa mengubah label. Dalam penelitian ini, beberapa transformasi yang diterapkan adalah sebagai berikut:

i) Resize

Transformasi pertama yang dilakukan adalah resize gambar menjadi ukuran 256×256 piksel. Ukuran gambar yang konsisten penting untuk memastikan gambar dapat diproses secara efektif oleh model. Dengan menggunakan resize, memastikan bahwa semua gambar memiliki ukuran yang seragam sebelum diproses lebih lanjut.

ii) Random Rotation

Untuk meningkatkan variasi gambar, juga dilakukan random rotation pada gambar. Merotasi gambar secara acak hingga 15 derajat.

Teknik ini membantu model agar tidak terlalu bergantung pada orientasi gambar, yang bisa terjadi pada gambar medis seperti X-Ray, di mana posisi objek mungkin tidak selalu konsisten.

iii) Random Horizontal Flip

Selain dilakukan random rotation, random horizontal flip digunakan untuk membalik gambar secara acak secara horizontal. Hal ini penting untuk membuat model lebih mengenal posisi objek yang bisa jadi muncul di berbagai sisi gambar, menambah variasi tanpa mengubah data yang ada.

iv) Center Crop

Setelah melakukan resize dan transformasi lainnya, gambar akan dipotong dengan center crop untuk ukuran 224×224 piksel. Crop ini berfungsi untuk mengambil bagian tengah gambar setelah resizing. Dengan crop ini, kita dapat memastikan bahwa area yang paling relevan dari gambar bagian tengah tetap terjaga, dan ukuran gambar yang digunakan tetap konsisten dengan model yang akan dilatih.

v) RGB to Grayscale

Data yang digunakan adalah gambar X-Ray yang umumnya berupa gambar grayscale (hitam-putih), maka perlu dilakukan pengubahan semua gambar menjadi grayscale dengan satu channel output. Ini akan memastikan bahwa gambar yang dimasukkan ke model

memiliki representasi yang konsisten dengan data asli, di mana hanya intensitas cahaya yang diukur, bukan warnanya.

vi) ToTensor

ToTensor digunakan untuk mengubah gambar menjadi tensor yang dapat diproses oleh model deep learning. Tensor adalah format data numerik yang bisa dipahami oleh PyTorch, dan memungkinkan untuk menggunakan GPU dalam pelatihan.

vii) Normalize

Terakhir, dilakukan normalisasi pada gambar, yang bertujuan untuk menyesuaikan distribusi nilai piksel gambar dengan rata-rata dan deviasi standar tertentu. Normalisasi ini sangat penting untuk membantu model belajar lebih cepat dan stabil. Untuk grayscale, nilai normalisasi menggunakan rata-rata dan deviasi standar yaitu 0.5.

4) Modeling

a. Build Pretrained DenseNet121

Pada tahap ini, arsitektur DenseNet121 digunakan sebagai pretrained model untuk klasifikasi Tuberkulosis (TB) vs Normal menggunakan citra X-ray. Model ini sebelumnya telah dilatih pada dataset ImageNet, sehingga memiliki kemampuan ekstraksi fitur yang kuat. Namun, karena citra X-ray yang digunakan dalam penelitian ini berupa grayscale (1 channel), maka dilakukan modifikasi pada first convolution layer agar dapat menerima input grayscale.

Selain itu, untuk memanfaatkan fitur yang telah dipelajari dari pretrained model, semua parameter pada DenseNet121 dibekukan (*frozen*), kecuali bagian classifier yang bertujuan untuk memanfaatkan fitur-fitur yang telah dipelajari dari pretrained model pada dataset ImageNet. Dengan cara ini, lapisan-lapisan awal yang berfungsi sebagai feature extractor tetap mempertahankan bobot yang telah dipelajari sebelumnya, sehingga tidak perlu dilatih ulang dari awal.

Bagian classifier dilakukan penyesuaian agar sesuai dengan tugas klasifikasi biner. Struktur baru classifier terdiri dari beberapa lapisan fully connected (FC) dengan ReLU activation, serta tambahan Dropout layers untuk mengurangi risiko overfitting. Setelah semua modifikasi dilakukan, model kemudian dipindahkan ke perangkat GPU atau CPU agar siap untuk proses pelatihan. Pada penelitian ini arsitektur model DenseNet121 yang digunakan adalah seperti pada Tabel 3.2 berikut.

Tabel 3.2 Arsitektur Model DenseNet121

No	Layer Type	Output Size	Detail
1	Input Image	1 x 224 x 224	Gray scale input (1 channel)
2	Conv	64 x 112 x 112	7 x 7 conv, stride 2, padding 3
3	MaxPool	64 x 56 x 56	3 x 3 max pool, stride 2, padding 1
4	Dense Block 1	256 x 56 x 56	6 x (1 x 1 conv + 3 x 3 conv), growth rate = 32
5	Transition Layer 1	128 x 28 x 28	1 x 1 conv, 2 x 2 avg pool, stride 2

6	Dense Block 2	512 x 28 x 28	12 x (1 x 1 conv, 3 x 3 conv)
7	Transition Layer 2	256 x 14 x 14	1 x 1 conv, 2 x 2 avg pool, stride 2
8	Dense Block 3	1024 x 14 x 14	24 x (1 x 1 conv, 3 x 3 conv)
9	Transition Layer 3	512 x 7 x 7	1 x 1 conv, 2 x 2 avg pool, stride 2
10	Dense Block 4	1024 x 7 x 7	16 x (1 x 1 conv, 3 x 3 conv)
11	Global Avg Pool	1024 x 1 x 1	7 x 7 global avg pooling
12	Fully Connected	2	Output binary (TB vs Normal)

b. Training Model

Sebelum proses pelatihan dilakukan, model DenseNet121 yang telah dimodifikasi perlu dikonfigurasi dengan sejumlah hyperparameter penting untuk mengarahkan proses training. Hyperparameter ini mencakup pemilihan fungsi loss, optimizer, nilai learning rate, jumlah epoch, batch size, serta penggunaan class weight untuk mengatasi ketidakseimbangan kelas.

Pada penelitian ini, digunakan fungsi loss *Binary Cross Entropy* karena tugas klasifikasi bersifat biner (TB vs Normal), serta optimizer Adam dengan learning rate sebesar 0.001 untuk mempercepat proses konvergensi. Selain itu, jumlah epoch ditetapkan sebanyak 25, sedangkan batch size ditentukan sebesar 32 untuk menjaga keseimbangan antara akurasi dan efisiensi komputasi. Konfigurasi ini bertujuan untuk memastikan proses pelatihan berjalan optimal dan model mampu belajar

representasi fitur dari data secara efektif sebelum dievaluasi pada data validasi dan pengujian. Untuk lebih jelasnya, konfigurasi hyperparameter yang digunakan dalam proses pelatihan disajikan pada Tabel 3.3 berikut.

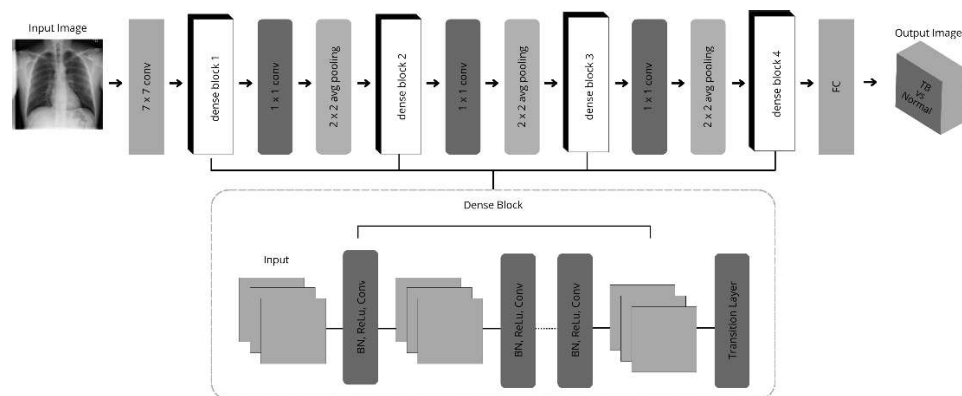
Tabel 3.3 Hyperparameter Pelatihan Model

No	Hyperparameter	Nilai
1	Loss Function	Binary Cross Entropy
2	Optimizer	Adam
3	Learning Rate	0.0001
4	Epochs	25
5	Batch Size	32

Setelah seluruh hyperparameter ditentukan sebagaimana ditunjukkan pada Tabel 3.3, proses pelatihan model dapat dilakukan. Pada tahap ini, model dijalankan selama 25 epoch dengan menggunakan data latih dan data validasi. Proses pelatihan terdiri dari dua langkah utama, yaitu forward pass dan backward pass.

Pada proses forward pass, input berupa citra X-ray grayscale akan diproses melewati seluruh lapisan dalam arsitektur DenseNet121, mulai dari convolutional layer, dense blocks, hingga fully connected layer yang telah dimodifikasi. Model akan menghasilkan output prediksi yang kemudian dibandingkan dengan label sebenarnya menggunakan fungsi loss. Hasil perbandingan ini menghasilkan nilai loss yang merepresentasikan seberapa jauh prediksi model dari nilai yang seharusnya.

Setelah itu, dilakukan backward pass dengan menggunakan teknik backpropagation untuk menghitung gradien dari loss terhadap parameter-parameter model. Optimizer Adam kemudian memanfaatkan gradien ini untuk melakukan pembaruan bobot model agar model semakin mampu mempelajari pola dari data. Proses ini dilakukan secara iteratif selama epoch berjalan, dengan tujuan untuk meminimalkan loss dan meningkatkan akurasi prediksi model. Skema alur pelatihan model dapat dilihat pada Gambar 3.4 berikut.



Gambar 3.4 Pelatihan Model DenseNet121

c. Evaluation Model

Setelah pelatihan model selesai, langkah selanjutnya adalah evaluasi untuk menilai performa model terhadap data validasi dan data uji. Evaluasi dilakukan menggunakan beberapa metrik, antara lain akurasi, confusion matrix, F1-score, dan AUC-ROC.

Akurasi digunakan untuk mengukur proporsi prediksi yang benar. Confusion matrix memberikan rincian jumlah prediksi benar dan salah dari

masing-masing kelas, termasuk True Positive (TP), True Negative (TN), False Positive (FP), dan False Negative (FN).

Untuk mengatasi ketidakseimbangan data, digunakan juga F1-score yang mempertimbangkan precision dan recall secara seimbang. Terakhir, AUC-ROC digunakan untuk menilai kemampuan model dalam membedakan antara kelas TB dan Normal pada berbagai threshold.

Penggunaan metrik-metrik ini bertujuan untuk memberikan gambaran menyeluruh terhadap performa model, khususnya dalam mendeteksi kasus Tuberkulosis secara akurat.

d. Saved Model

Setelah proses pelatihan selesai, model disimpan dalam dua format utama, yaitu .pth dan .onnx, guna mendukung fleksibilitas penggunaan di berbagai platform. Format .pth digunakan untuk menyimpan seluruh model PyTorch, termasuk arsitektur dan bobot yang telah dipelajari. Penyimpanan dilakukan menggunakan fungsi `torch.save()`, sehingga model dapat dimuat kembali secara langsung di lingkungan PyTorch tanpa perlu membangun ulang struktur model.

Sementara itu, format .onnx (Open Neural Network Exchange) digunakan untuk membuat model lebih portabel dan kompatibel dengan berbagai framework lain di luar PyTorch, seperti TensorFlow atau ONNX Runtime. Konversi ke ONNX dilakukan menggunakan `torch.onnx.export()`

dengan input dummy berukuran [1, 1, 224, 224] yang merepresentasikan gambar grayscale berukuran 224 x 224 piksel.

Dengan menyimpan model dalam kedua format ini, proses deployment, baik dalam platform berbasis PyTorch maupun lintas platform, dapat dilakukan dengan lebih mudah dan efisien.

5) Implementasi Sistem

Implementasi sistem dilakukan dengan pendekatan arsitektur terpisah antara frontend dan backend. Untuk sisi frontend, digunakan framework Next.js yang bertanggung jawab dalam membangun antarmuka pengguna, sedangkan pada sisi backend, digunakan FastAPI sebagai framework yang menangani permintaan prediksi dari frontend dan menjalankan proses inferensi model.

Model deteksi yang telah dilatih sebelumnya disimpan dalam dua format, yaitu .pth dan .onnx. Pada sistem ini, yang diimplementasikan adalah model dalam format ONNX karena lebih efisien dan kompatibel untuk deployment. Model ONNX digunakan oleh backend FastAPI untuk melakukan prediksi terhadap citra yang dikirimkan dari frontend. Setelah dilakukan preprocessing, citra X-ray tersebut dimasukkan ke model ONNX, dan hasil prediksinya dikembalikan ke frontend untuk ditampilkan ke pengguna.

6) Pengujian Sistem

Pengujian sistem bertujuan untuk memastikan bahwa seluruh komponen dalam sistem dapat bekerja secara terintegrasi dan sesuai dengan alur penggunaan sebenarnya. Dalam penelitian ini, pengujian dilakukan

menggunakan pendekatan End-to-End Testing (E2E), dengan mensimulasikan interaksi pengguna dari awal hingga akhir proses dalam sistem berbasis web.

Pendekatan ini dipilih karena sistem yang dikembangkan mencakup proses yang kompleks dan saling terhubung, mulai dari antarmuka pengguna (frontend dengan Next.js), komunikasi API (backend FastAPI), hingga pemanggilan model deteksi Tuberkulosis yang telah disimpan dalam format ONNX. Dengan demikian, E2E testing dapat memberikan gambaran menyeluruh terhadap pengalaman pengguna dan kestabilan sistem.

D. Teknik Pengembangan Sistem

Teknik pengembangan sistem yang diterapkan dalam penelitian ini adalah studi pustaka. Studi pustaka dilakukan dengan mengumpulkan dan mempelajari berbagai referensi yang relevan dari sumber-sumber terpercaya seperti buku, e-book, jurnal ilmiah, serta informasi dari internet. Tujuannya adalah untuk memperoleh pemahaman yang mendalam mengenai topik penelitian, khususnya dalam hal pengolahan citra medis, deteksi penyakit Tuberkulosis menggunakan metode deep learning, serta pengembangan sistem berbasis web.

Informasi yang diperoleh dari studi pustaka ini menjadi dasar dalam perancangan model, pemilihan arsitektur yaitu DenseNet121, teknik preprocessing citra, pemilihan metode evaluasi, hingga implementasi model ke dalam sistem berbasis web.